

```

enum class Unit {
    UNKNOWN = 0,

    // Basic types.
    KEYWORD = 1 << 0, // generic keyword; fetch as <int>
    STRING = 1 << 1,  // generic string; fetch as <String>
    COLOUR = 1 << 2,   // colour; fetch as <Colourb>
    RATIO = 1 << 3,    // ratio defined as x/y; fetch as <Vector2f>

    // Numeric values.
    NUMBER = 1 << 4,   // number unsuffixed; fetch as <float>
    PERCENT = 1 << 5,  // number suffixed by '%'; fetch as <float>
    PX = 1 << 6,       // number suffixed by 'px'; fetch as <float>

    // Context-relative values.
    DP = 1 << 7, // density-independent pixel; number suffixed by 'dp'; fetch as <float>
    VW = 1 << 8, // viewport-width percentage; number suffixed by 'vw'; fetch as <float>
    VH = 1 << 9, // viewport-height percentage; number suffixed by 'vh'; fetch as <float>
    X = 1 << 10, // dots per px unit; number suffixed by 'x'; fetch as <float>

    // Font-relative values.
    EM = 1 << 11, // number suffixed by 'em'; fetch as <float>
    REM = 1 << 12, // number suffixed by 'rem'; fetch as <float>

    // Values based on pixels-per-inch.
    INCH = 1 << 13, // number suffixed by 'in'; fetch as <float>
    CM = 1 << 14,   // number suffixed by 'cm'; fetch as <float>
    MM = 1 << 15,   // number suffixed by 'mm'; fetch as <float>
    PT = 1 << 16,   // number suffixed by 'pt'; fetch as <float>
    PC = 1 << 17,   // number suffixed by 'pc'; fetch as <float>
    PPI_UNIT = INCH | CM | MM | PT | PC,

    // Angles.
    DEG = 1 << 18, // number suffixed by 'deg'; fetch as <float>
    RAD = 1 << 19, // number suffixed by 'rad'; fetch as <float>

    // Values tied to specific types.
    TRANSFORM = 1 << 20, // transform; fetch as <TransformPtr>, may be empty
    TRANSITION = 1 << 21, // transition; fetch as <TransitionList>
    ANIMATION = 1 << 22, // animation; fetch as <AnimationList>
    DECORATOR = 1 << 23, // decorator; fetch as <DecoratorsPtr>
    FILTER = 1 << 24, // decorator; fetch as <FiltersPtr>
    FONTEFFECT = 1 << 25, // font-effect; fetch as <FontEffectsPtr>
    COLORSTOPLIST = 1 << 26, // color stop list; fetch as <ColorStopList>
    BOXSHADOWLIST = 1 << 27, // shadow list; fetch as <BoxShadowList>

```

```

    LENGTH = PX | DP | VW | VH | EM | REM | PPI_UNIT,
    LENGTH_PERCENT = LENGTH | PERCENT,
    NUMBER_PERCENT = NUMBER | PERCENT,
    NUMBER_LENGTH_PERCENT = NUMBER | LENGTH | PERCENT,
    DP_SCALABLE_LENGTH = DP | PPI_UNIT,
    ANGLE = DEG | RAD,
    NUMERIC = NUMBER_LENGTH_PERCENT | ANGLE | X
};

// Type alias hint for indicating that the value can contain multiple units OR-ed together.
using Units = Unit;

// OR operator for combining multiple units where applicable.
inline Units operator|(Units lhs, Units rhs)
{
    using underlying_t = std::underlying_type<Units>::type;
    return static_cast<Units>(static_cast<underlying_t>(lhs) | static_cast<underlying_t>(rhs));
}

// AND operator for combining multiple units where applicable.
inline Units operator&(Units lhs, Units rhs)
{
    using underlying_t = std::underlying_type<Units>::type;
    return static_cast<Units>(static_cast<underlying_t>(lhs) & static_cast<underlying_t>(rhs));
}

// Returns true if any unit is set.
inline bool Any(Units units)
{
    return units != Unit::UNKNOWN;
}

} // namespace Rml
#endif

```